STALL ALERTER

Production-Grade Product & Technical Implementation Blueprint

AI-centric hiring-pipeline monitoring, stall detection, workflow automation, and recruiter decision support

Purpose: A build-ready blueprint for moving from a validated MVP to a secure, reliable, deployable production application. This document specifies architecture, workflows, data, APIs, AI controls, integrations, testing, deployment, and operations.

Version 2.0 • Updated October 2026 • Implementation specification, not proof of a deployed system

1. Executive summary

Stall Alerter helps recruiting teams identify candidates whose hiring process appears stuck, understand why, and take the next appropriate action. It ingests candidate and job-stage data, normalizes it, calculates time-in-stage and SLA risk, uses AI to summarize evidence and draft recommended follow-ups, and automates approved operational workflows.

The system should be AI-centric, but not AI-only for safety-critical facts. Deterministic rules must own timestamps, stage transitions, SLA calculations, permissions, and delivery state. AI should interpret unstructured job descriptions and notes, explain evidence, summarize context, draft messages, and help users configure workflows. This hybrid approach is more reliable than asking a language model to make every decision.

Production-ready means more than writing code: security controls, integration contracts, observability, data retention, backups, recovery, tests, runbooks, staged releases, and operational ownership must be implemented and verified.

> **Important boundary: Stall Alerter is a recruiting operations tool, not an autonomous hiring decision-maker. It must not reject, rank, or advance people solely from AI output. A human remains accountable for candidate evaluation and consequential communications.**

2. Product definition and outcomes

Primary users

- **Recruiter:** monitors active candidates, sees overdue stages, reviews context, and sends approved follow-ups.
- **Recruiting operations lead:** configures stage SLAs, escalation rules, ownership, and reporting.
- **Hiring manager:** receives actionable reminders and provides missing feedback.
- **Workspace administrator:** manages access, integrations, retention, and audit trails.

Core outcomes

- Reduce time candidates remain unattended in a stage.
- Make stalled or at-risk cases visible with clear, explainable reasons.
- Reduce repetitive recruiter coordination through safe, configurable automation.
- Improve data quality and handoff accountability.
- Make AI-generated summaries and drafts traceable to source evidence.

Non-goals

- No automated candidate rejection, hiring decision, or opaque employability score.
- No scraping of restricted systems or bypassing ATS access controls.
- No sending external candidate messages without the configured authorization and review policy.
- No claim that AI can infer facts absent from the source data.

3. End-to-end user workflow

Step	System behavior	User-visible result
1. Connect/import	Admin connects an authorized ATS or uploads CSV/XLSX template; file is scanned and validated.	Import preview, row errors, duplicate warnings.
2. Normalize	Map source fields to canonical candidate, job, application, stage, owner, and timestamp fields.	Consistent pipeline records with data-quality status.
3. Detect stalls	Compute stage age, business-hours age if configured, SLA breach, missing feedback, and stale sync.	Candidate/application appears as on-track, at-risk, or stalled.
4. Enrich with AI	Parse job description and notes; summarize only relevant evidence with source references.	Short explanation, missing information, and confidence/limitations.
5. Prioritize operations	Use configurable urgency factors such as SLA breach, days idle, missing owner, and candidate response pending.	Explainable action queue; not a candidate merit ranking.
6. Recommend action	Generate next-step suggestions and draft an email/task using approved templates.	Editable draft with reason, evidence, and approval state.
7. Approve/execute	User approves, or a narrowly scoped pre-approved rule executes a safe internal task.	Audit event, delivery status, retry state.
8. Monitor/learn	Track resolution, override reasons, false positives, latency, costs, and integration failures.	Dashboard and quality/cost reports.

Every automated action must be idempotent, auditable, permission-checked, and reversible where possible. If input data is stale or ambiguous, the system should show a warning and avoid external actions.

4. Functional requirements

MVP requirements

- CSV import with a downloadable template, field mapping, validation, preview, and import summary.
- Configurable stage definitions and per-stage SLA thresholds.
- Candidate/application table with filters for stalled, at-risk, owner, job, stage, and age.
- Application detail with stage timeline, source fields, stall reason, and audit history.
- AI-assisted job description extraction, evidence-based summary, and next-step draft.
- Manual approval before candidate-facing email; internal reminders may be rule-enabled.
- Basic dashboard, authentication, role checks, error states, and test fixtures.

Production v1 requirements

- Multi-tenant workspaces with enforced tenant isolation and role-based access control.
- ATS connector framework with webhook ingestion, polling fallback, rate limits, and sync checkpoints.
- Email/calendar/task integrations with OAuth, token refresh, revocation, and least privilege.
- Durable background jobs, retries, dead-letter queue, deduplication, and replay tooling.
- Admin configuration, audit export, retention controls, deletion workflows, and incident procedures.
- Service health, metrics, logs, traces, alerting, backup/restore, disaster recovery, and release rollback.

- AI provider abstraction, structured outputs, prompt/version tracking, model fallbacks, evaluation suite, and budget controls.

5. Architecture

Recommended starting architecture: a modular monolith with asynchronous workers. This is easier to build and operate than premature microservices while still separating responsibilities. Split services only when measured scale or team ownership justifies it.

Layer	Recommended component	Responsibilities
Web UI	Next.js/TypeScript or equivalent	Pipeline, import wizard, candidate detail, rules, integrations, approvals, admin.
API	Python FastAPI	Authentication integration, tenant-scoped APIs, validation, authorization, orchestration.
Primary database	PostgreSQL	Canonical data, tenant boundaries, workflow state, audit records, configuration.
Queue/cache	Redis + Celery/RQ or managed queue	Imports, scheduled SLA checks, AI jobs, outbound delivery, retry handling.
Object storage	S3-compatible private bucket	Uploaded files, exports, attachments where required; encryption and lifecycle rules.
AI gateway	Internal provider adapter	Model selection, prompt versions, structured schema validation, redaction, cost/latency tracking.
Observability	OpenTelemetry + managed metrics/logging	Request traces, job traces, dashboards, alert rules, error reporting.
Deployment	Docker + managed container platform	Repeatable builds, health checks, secrets, autoscaling, rolling deployment.

Logical flow

```
ATS / CSV / Webhooks
  -> ingestion API -> validation + canonical mapping -> PostgreSQL
  -> scheduled rule engine -> stall events -> durable queue
  -> AI enrichment / draft generation -> policy + schema validation
  -> approval queue -> email/task connector -> delivery receipt + audit log
  -> dashboards, metrics, alerts, and quality evaluation
```

Key architectural rule

The database and rule engine are the source of truth for operational state. The language model is a bounded assistant: it does not directly write arbitrary database state, send arbitrary messages, or execute tools without application policy checks.

6. Canonical data model

Entity	Important fields / purpose
Workspace	id, name, plan, timezone, retention_policy, created_at
User / Membership	id, workspace_id, identity_provider_id, role, status
Job	id, workspace_id, external_job_id, title, department, location, status, description, source
Candidate	id, workspace_id, external_candidate_id, minimal contact fields, source, consent/legal basis metadata
Application	id, workspace_id, candidate_id, job_id, current_stage_id, owner_user_id, status, last_activity_at, source_updated_at
StageDefinition	id, workspace_id, name, order_index, SLA duration, business-hours calendar, escalation policy
StageEvent	id, application_id, from_stage, to_stage, occurred_at, actor, source_event_id, raw_source_ref
StallEvent	id, application_id, detected_at, severity, reason_codes, rule_version, resolved_at, resolution_reason
AIArtifact	id, workspace_id, application_id, artifact_type, model/provider, prompt_version, structured_output, source_refs, created_at
WorkflowRule	id, workspace_id, trigger, conditions, actions, approval_mode, enabled, version
ActionExecution	id, rule_id, idempotency_key, status, attempt_count, scheduled_at, executed_at, error_code
IntegrationConnection	id, workspace_id, provider, encrypted_token_ref, scopes, sync_cursor, status, last_success_at
AuditEvent	id, workspace_id, actor, action, entity_type, entity_id, timestamp, request_id, redacted_diff
AIUsageRecord	id, workspace_id, provider, model, tokens, latency_ms, estimated_cost, outcome

Use UUIDs or equivalent non-guessable identifiers. Apply workspace_id consistently, enforce tenant scope in every repository query, and add database constraints/indexes for common filters. Store only the minimum candidate data needed. Keep raw ATS payloads short-lived and access-controlled.

7. Stall detection and operational priority

Stall logic

A stall is a workflow state, not an AI opinion. Example rule: an active application is stalled when elapsed time since the most recent valid stage-entry event exceeds the configured SLA, excluding configured paused states and holidays.

```
stage_age = now_in_workspace_timezone - valid_stage_entered_at
is_stalled = active AND stage_age > stage_sla AND NOT paused
is_at_risk = active AND stage_age >= stage_sla * risk_threshold
```

Use UTC for stored timestamps and the workspace timezone for display and business-hour calculations. Define how missing stage-entry timestamps, reopened applications, duplicate events, paused applications, and out-of-order ATS events are handled.

Operational priority

Calculate a transparent operations priority from SLA breach size, time idle, candidate waiting state, missing owner, and business impact. Display the factors and allow administrators to tune weights. Do not label this score as candidate quality, employability, or likelihood of success.

False-positive controls

- Deduplicate events using source event IDs and idempotency keys.
- Do not trigger on records with stale or incomplete sync state without a warning.
- Support pause reasons, holidays, stage-specific calendars, and manual resolution.
- Allow users to mark false positives with a reason and include them in evaluation reports.
- Version every SLA/rule configuration and record which version caused a stall.

8. AI-centric capabilities and safeguards

AI use cases

Capability	Input	Output	Control
JD parsing	Job description text	Structured role requirements and must-have/nice-to-have labels	Schema validation; editable extraction; no inferred facts.
Evidence summary	Permitted resume/application notes and source references	Concise evidence-linked summary	Quote/reference source IDs; label missing evidence; avoid protected-attribute inference.
Context summary	Stage history, feedback status, notes	Why the workflow may be blocked	Distinguish observed facts from hypotheses.
Action recommendation	Stall reason, stage, workflow policy	Recommended operational next step	Allowed-action list; reason and confidence/uncertainty.
Message draft	Approved template + relevant facts	Editable follow-up draft	Human approval by default; PII minimization; no invented promises.
Natural-language rule setup	Admin request	Draft workflow configuration	Preview, validation, permission check, explicit save/enable.
Analytics assistant	Aggregated and authorized metrics	Answers and trend summaries	Tenant-scoped query layer; no unrestricted SQL execution.

AI execution pipeline

- Collect only authorized, relevant fields; redact unnecessary personal data.
- Build a typed request from trusted application state, not directly from user-controlled free text.
- Call a provider through a versioned adapter with timeout, retry policy, token limit, and budget cap.
- Require JSON/schema-constrained output; reject malformed or out-of-policy responses.
- Validate source references against known records; mark unsupported statements as unsupported.
- Apply business policy and permissions after model output; the model cannot override policy.
- Store prompt/model version, source references, latency, cost, and outcome; avoid logging raw sensitive prompts by default.
- Use human review for consequential or candidate-facing actions; provide a safe fallback if AI is unavailable.

Prompt injection and untrusted data

Treat job descriptions, resumes, emails, notes, and ATS text as untrusted data. Explicitly instruct the model to analyze the content rather than follow instructions inside it. Never put secrets in prompts. Do not let model output select arbitrary URLs, SQL, shell commands, recipients, or tools. Use allow-listed actions and validate all recipients against application state.

AI evaluation

- Create a representative, permissioned test set with expected structured outputs and edge cases.
- Measure factuality, evidence coverage, unsupported-claim rate, schema-validity rate, recommendation usefulness, and unsafe-action rate.
- Run regression evaluations when prompts, models, data mappings, or policies change.
- Track cost and latency by workflow and tenant; set per-workspace quotas and global circuit breakers.
- Compare models on quality and cost before switching; maintain a deterministic fallback for stall detection.

9. Automation engine

Triggers

- Stage entered or changed; application updated; candidate response received.
- SLA threshold reached; stage overdue; owner missing; hiring-manager feedback pending.
- Scheduled daily/weekly reconciliation; ATS sync failed; integration token expired.

Actions

- Create an internal task or reminder.
- Notify the assigned recruiter or hiring manager.
- Generate an AI summary and draft a follow-up.
- Escalate after configurable time and repeat limits.
- Send an external message only under the configured approval policy.

Execution semantics

Use a durable queue and an action-execution record. Every action has a unique idempotency key, bounded retries with exponential backoff and jitter, timeout, status, and terminal failure handling. External delivery is at-least-once in many systems; protect against duplicates and reconcile provider receipts. Add a dead-letter queue and admin replay tool.

State	Meaning
pending_approval	Draft exists; user action required.
queued	Policy checks passed and execution is scheduled.
running	Worker has claimed the job with a lease.
succeeded	Provider or internal system confirmed completion.
retrying	Transient failure; another attempt is scheduled.
failed_terminal	Retry budget exhausted; admin-visible failure.
cancelled	User or policy cancelled before completion.

10. API contracts

Method / route	Purpose	Access / notes
POST /v1/imports	Upload/create import job	Workspace admin/recruiter; file size/type limits.
GET /v1/imports/{id}	Read import progress and errors	Workspace-scoped.
GET /v1/applications	List/filter applications and stall state	Cursor pagination; tenant-scoped.
GET /v1/applications/{id}	Application detail and timeline	Authorized workspace member.
POST /v1/applications/{id}/analyze	Request AI context summary	Rate-limited async job; returns job ID.
POST /v1/applications/{id}/drafts	Create follow-up draft	Does not send automatically.
POST /v1/drafts/{id}/approve	Approve a draft/action	Role and policy check; audit event.
POST /v1/rules	Create/update workflow rule	Admin; version and dry-run required.
POST /v1/rules/{id}/dry-run	Simulate rule against selected data	No side effects.
POST /v1/integrations/{provider}/connect	Start OAuth/connection flow	Admin; state/PKCE/CSRF validation.
POST /v1/webhooks/{provider}	Receive provider events	Signature validation, replay protection.
GET /v1/audit-events	Search audit trail	Restricted; filters and export controls.
GET /v1/health/live	Process liveness	No secrets or internal details.
GET /v1/health/ready	Dependency readiness	Used by deployment health checks.

All write endpoints should validate schemas, enforce authorization, use request IDs, return stable error shapes, and be covered by OpenAPI contract tests. Apply rate limits and payload-size limits. Never trust a workspace ID supplied by the client without checking membership.

11. Integration framework

ATS integration requirements

- Use official APIs and documented OAuth scopes; verify each provider's current API and commercial access terms.
- Support incremental sync, cursor persistence, rate-limit handling, pagination, retries, webhook signature validation, and periodic reconciliation.
- Map external IDs to internal IDs; retain source_updated_at and last_synced_at.
- Handle deletion, archived jobs, changed stages, reopened applications, duplicate webhooks, and late events.
- Expose connection health, last successful sync, lag, failures, and a reconnect action.

Email, calendar, and task tools

Start with one email provider and one internal task/notification channel. Use OAuth or managed identity, minimum scopes, token rotation, delivery receipts, unsubscribe/communication preferences where applicable, and a clear sender identity. Calendar access should be read/write only if the product truly needs it.

Integration rollout

Begin with CSV import as the reliable baseline, then add one ATS connector behind a common adapter interface. Do not promise universal ATS compatibility until each connector has been built, tested, and approved.

12. Security, privacy, and governance

- Authentication via a mature identity provider; MFA for admins; short-lived sessions and secure cookies.
- Role-based permissions plus tenant isolation; test cross-tenant access attempts explicitly.
- TLS in transit, managed encryption at rest, secrets manager, token encryption, and key rotation.
- Least-privilege service accounts, separate development/staging/production credentials, and no secrets in source control.
- Audit logs for access, exports, rule changes, approvals, sends, and deletion actions.
- Data minimization, retention schedules, export/deletion workflows, backup retention, and documented legal basis/consent where required.
- PII redaction in logs and AI requests; access-controlled raw payloads; secure file scanning.
- Dependency/SBOM scanning, static analysis, container scanning, vulnerability patch policy, and incident response plan.
- Privacy and employment-law review for each operating jurisdiction; do not infer or use protected attributes for decisions.

Security posture must be reviewed by qualified professionals before handling real candidate data. Compliance is not established merely by adding a checklist to the product.

13. Reliability, performance, and operations

Area	Production expectation
Availability	Define an initial SLO, e.g. 99.5% monthly API availability, and revise from real usage.
Latency	Set route-specific p95/p99 budgets; async AI/import work returns job IDs instead of blocking requests.
Queue health	Monitor oldest queued job, throughput, retries, worker saturation, and dead-letter count.
Data integrity	Transactional updates, uniqueness constraints, idempotent consumers, and periodic reconciliation.
Backups	Automated encrypted backups; test restore regularly; define RPO/RTO before launch.
Observability	Structured logs, metrics, traces, correlation IDs, dashboards, and actionable alerts.
Incident response	On-call owner, severity definitions, escalation path, status communication, and postmortems.
Release safety	Staging, migrations with backward compatibility, canary/rolling release, rollback procedure.
Cost	Per-tenant AI usage budgets, model limits, queue concurrency caps, and alert thresholds.

Initial SLO suggestions

- API availability: 99.5% monthly for production v1, excluding agreed maintenance windows.
- Scheduled stall detection: 95% of active applications recalculated within 10 minutes of their scheduled interval.
- Import reliability: every import ends in success, partial success with row-level errors, or a clear terminal failure.
- Action reliability: no silent failure; every queued action reaches a terminal state or remains visibly retrying.
- AI reliability: schema-valid output target measured continuously; AI outage never disables deterministic stall detection.

14. Testing and release gates

Test layer	What must be tested
Unit	SLA boundaries, business calendars, paused stages, priority factors, mappings, permissions.
Integration	PostgreSQL transactions, queue, object storage, AI provider adapter, email provider, ATS sandbox.
Contract	OpenAPI schemas, webhook signatures, provider payload fixtures, pagination and error semantics.
End-to-end	Import -> stall detection -> AI draft -> approval -> send/task -> audit record.
Security	Tenant escape, IDOR, CSRF, XSS, injection, SSRF, file upload abuse, token leakage, privilege escalation.
Resilience	Provider timeout, rate limit, queue outage, duplicate webhook, worker crash, DB failover, retry storm.
AI evaluation	Hallucinations, missing evidence, prompt injection, biased language, schema failures, unsafe action proposals.
Performance	Expected data volume, large imports, dashboard filters, queue backlogs, concurrency, load and soak tests.
Accessibility	Keyboard navigation, labels, contrast, screen reader checks, clear status/error messaging.

Release gate — do not launch until

- Critical/high security findings are resolved or formally risk-accepted.
- Restore test, rollback test, alert test, and incident runbook walkthrough pass.
- All critical user journeys pass in staging with synthetic data.
- AI output has an evaluation baseline and human approval policy is enforced in code.
- No cross-tenant leakage is observed in automated authorization tests.
- Integration rate limits, token expiry, retries, and reconnection have been tested.
- Privacy notice, terms, retention policy, support contact, and data processing review are complete.

15. Deployment and environment strategy

Environments

- Local: Docker Compose, synthetic data, mock AI provider, fake integrations.
- CI: lint, type checks, unit tests, security/dependency scans, build, and migration validation.
- Staging: production-like services, synthetic or masked data, ATS sandbox, email sink, full E2E tests.
- Production: managed database, private networking, secrets manager, restricted access, backups, alerts, and documented rollback.

Example repository layout

```
stall-alerter/
  apps/web/                # UI
  services/api/            # FastAPI routes and auth
  services/worker/         # imports, scheduled jobs, AI, delivery
  packages/contracts/      # schemas and shared API types
  integrations/ats/        # connector adapters
  integrations/email/      # email provider adapter
  ai/prompts/              # versioned prompt templates
  ai/evals/                # datasets and evaluation runners
  migrations/              # database migrations
  infra/                   # IaC, container, deployment manifests
  tests/unit/
  tests/integration/
  tests/e2e/
  docs/runbooks/
  .github/workflows/       # CI/CD
```

Release process

- Build immutable container images and pin dependencies.
- Run migrations as a controlled release step with backward-compatible schema changes.
- Deploy to staging and run smoke, E2E, security, and AI evaluation suites.
- Release to a small canary/workspace cohort; watch errors, latency, queue lag, and AI costs.
- Expand rollout only after the observation window passes; preserve rollback instructions.

16. Observability and admin dashboards

- Business metrics: active applications, stalled count, overdue duration, time-to-resolution, false-positive rate, follow-up completion.
- Integration metrics: sync lag, last success, error rate, token expiry, rate-limit events, webhook delay.
- AI metrics: latency, schema-valid rate, unsupported-claim rate, review/override rate, provider failures, cost per action.
- System metrics: API error rate, p95 latency, DB pool saturation, queue depth, retry count, worker health.
- Audit dashboard: rule changes, approvals, sent messages, exports, permission changes, and deletion requests.

Avoid logging full resumes, emails, API tokens, or unredacted candidate details in general application logs.

17. Product analytics and success metrics

Metric	Definition / purpose
Stall detection precision	Share of flagged cases confirmed as genuinely stalled by reviewers.
False-positive rate	Flagged cases marked not stalled divided by reviewed flagged cases.
Time to action	Time from stall detection to a meaningful task, feedback, or approved outreach.
Time in stage	Median and percentile stage duration, segmented by job/stage without unfair candidate scoring.
Automation completion	Actions completed successfully / actions attempted.
Human override rate	Share of AI drafts/recommendations materially edited or rejected.
AI unsupported-claim rate	Sampled outputs with unsupported factual claims.
Cost per resolved stall	AI + infrastructure + integration cost divided by resolved stalls.
Integration freshness	Age of latest successful source sync.
Adoption	Weekly active recruiters, reviewed alerts, completed actions, and retained workspaces.

18. Implementation roadmap

Phase	Build scope	Exit criteria
Phase 0 — discovery	Confirm target ATS, sample exports, stages, SLA policy, user roles, privacy constraints.	Approved requirements, test dataset, workflow maps, threat model.
Phase 1 — working MVP	Auth, workspace model, CSV import, PostgreSQL, stall rules, dashboard, AI summary/draft, approval, audit.	End-to-end flow works with synthetic data; core tests pass.
Phase 2 — pilot	One ATS connector, notifications/tasks, rule dry-run, basic monitoring, user feedback.	Pilot users confirm usefulness; false-positive and data quality issues measured.
Phase 3 — production v1	Tenant isolation hardening, durable queue, retries, security review, backups/restore, CI/CD, admin and retention.	Release gates pass; on-call/runbooks and recovery tested.
Phase 4 — scale	More connectors, analytics assistant, advanced workflow builder, cost optimization, enterprise controls.	Scale/load evidence supports capacity; each new connector independently certified.

The timeline depends on developer capacity, ATS access, provider approvals, security requirements, and the amount of code already built. Do not promise a production date before those dependencies are validated.

19. Suggested delivery plan

Sprint	Deliverables
Sprint 1	Product contracts, schema, auth/workspace boundary, CSV template/import preview, CI foundation.
Sprint 2	Canonical application model, stage timeline, SLA engine, stall queue, configurable stage thresholds.
Sprint 3	AI provider adapter, JD parsing, evidence-linked summaries, draft generation, schema/policy validation.
Sprint 4	Approval workflow, audit events, background jobs, retries, notifications, admin controls.
Sprint 5	ATS connector prototype, webhook/polling sync, monitoring dashboards, security and integration tests.
Sprint 6	Staging release, load/security/AI evaluation, restore/rollback exercises, pilot fixes, production go/no-go.

This is a planning sequence, not a guarantee that six sprints are sufficient. Each sprint ends with a demo, tests, documented limitations, and a decision on whether to proceed.

20. Sample workflow rule

```
Rule: Hiring-manager feedback overdue
Trigger: application remains in 'Interview Feedback' stage
Condition: configured SLA is exceeded AND application is active AND sync is fresh
Action 1: create internal task for assigned hiring manager
Action 2: notify recruiter with stage age and source timestamp
Action 3: ask AI to draft a neutral reminder from an approved template
Approval: recruiter approval required before external email
Safety: no candidate status change; no rejection/advancement decision
Reliability: idempotency key = application_id + rule_version + SLA_window
Audit: record trigger, condition results, approver, provider receipt, and outcome
```

A dry-run mode must show which records would be affected before a rule is enabled. Enabling an external communication rule should require an explicit authorized user action.

21. Example AI output contract

```
{
  "summary": "Interview feedback is still pending.",
  "observed_facts": [
    {"claim": "Application has been in Interview Feedback for 4 days",
     "source_ref": "stage_event_123"}
  ],
  "unknowns": ["No feedback due date was supplied"],
  "recommended_operational_action": "Request feedback from the assigned interviewer",
  "draft_message": "Hi, could you please share feedback when you have a chance?",
  "requires_human_approval": true,
  "limitations": ["Stage timestamp may be delayed if the ATS sync is stale"]
}
```

The API must validate this output against a schema and confirm that each source_ref exists and is authorized. The example is illustrative and should be adapted to the product's actual schema.

22. Runbooks required before production

- ATS sync outage: pause unsafe actions, reconnect credentials, replay from cursor, reconcile records.
- AI provider outage: keep stall detection running, queue or disable AI enrichment, communicate degraded state.
- Email delivery failure: retry within policy, inspect provider receipt, prevent duplicate sends, escalate terminal failures.

- Queue backlog: inspect oldest job and worker health, scale within budget, control retries, check poison messages.
- Suspected data exposure: contain access, rotate credentials, preserve evidence, follow incident notification policy.
- Bad release: stop rollout, roll back application image, verify migration compatibility, reconcile queued work.
- Data deletion request: verify authorization, apply retention/legal hold rules, delete or anonymize relevant records, log outcome.

23. Open decisions that must be resolved

- Which ATS is first: Greenhouse, Lever, Workday, Zoho Recruit, another platform, or CSV only?
- Which AI provider(s), region, data-retention terms, and contractual protections are acceptable?
- Will candidate-facing messages require human approval in all cases or only certain approved workflows?
- What stage SLA defaults, business hours, holidays, paused states, and escalation policies apply?
- What are expected workspace count, application volume, import size, and availability needs?
- Which jurisdiction(s) and employment/privacy requirements apply to the first launch?
- What is the approved budget for cloud, AI, email, observability, and ATS access?

These choices affect design and cannot safely be guessed. The architecture is intentionally provider-adaptable until these decisions are confirmed.

24. Definition of done

- A new workspace can be created and secured; users can be assigned roles.
- A recruiter can import a supported file and understand every rejected/accepted row.
- A candidate application can be traced from source data through stage history to a stall reason.
- The SLA engine behaves correctly for boundaries, paused states, timezones, holidays, and duplicate events.
- AI summaries link claims to source evidence, identify unknowns, and pass schema/policy validation.
- Workflow rules can be dry-run, versioned, enabled, paused, and audited.
- External actions obey approval policy and cannot be triggered twice by duplicate events.
- Integration outages, provider rate limits, retries, and token expiry are visible and recoverable.
- Tenant isolation, deletion/retention, backups, restore, incident response, and rollback are tested.
- Dashboards and alerts cover product outcomes, security signals, reliability, AI quality, and cost.
- The release has passed documented go/no-go criteria and an authorized owner accepts operational responsibility.

25. What this document does and does not provide

Provides: a production-oriented product specification, architecture, workflow definitions, data model, API inventory, AI strategy, integration expectations, security/privacy controls, test plan, deployment strategy, metrics, roadmap, and release criteria.

Does not by itself provide: a complete executable codebase, valid credentials, configured ATS/email accounts, cloud resources, approved legal terms, a deployed URL, or evidence that production tests have passed.

To turn this blueprint into the actual product, implementation must proceed in the repository, with the target ATS and AI/email providers chosen, credentials configured securely, tests run, and deployment/operations verified.

The product should be described as production-ready only after the release gates in this document are demonstrably passed.

Appendix A — Minimum engineering checklist

- Repository protections, code review, branch checks, dependency lockfiles, secret scanning.
- Typed request/response schemas and generated API clients where practical.
- Database migrations reviewed and tested against realistic data volumes.
- All async tasks have idempotency, timeout, bounded retry, visibility, and terminal states.
- All AI prompts have owners, versions, test cases, and release notes.
- Every external side effect has authorization, audit logging, and a defined approval policy.
- No production candidate data in local fixtures; use synthetic or approved masked data.
- Security, privacy, reliability, and AI evaluation results attached to release review.

Appendix B — Recommended technical defaults

Concern	Default direction
Backend	FastAPI, Pydantic, SQLAlchemy, Alembic; pin and regularly update dependencies.
Database	Managed PostgreSQL with encrypted storage, point-in-time recovery, and connection pooling.
Jobs	Celery/RQ with Redis for an early modular monolith, or managed durable queue for stricter reliability.
Frontend	TypeScript app with accessible components and generated types from OpenAPI.
AI	Provider adapter, structured outputs, prompt registry, evaluation harness, budgets and fallback.
Files	Private object storage, signed short-lived URLs, file size/type checks, malware scanning.
Infrastructure	Docker, infrastructure as code, isolated environments, secrets manager, private networking.
CI/CD	Lint, type check, tests, scans, build, migration checks, staging E2E, approval-gated production deploy.